



MODUL TEORI

Disusun Oleh:

Aprilia Kurnianti, S.T., M.Eng.

Dosen Pengampuh Matkul IBD

DAFTAR ISI

Materi 6 TRIGGER, IDENTITY, DAN SEQUENCE	3
Pendahuluan.....	3
A. Deskripsi	3
B. IDENTITY	3
C. SEQUENCE.....	5
D. TRIGGER	8

Materi 6

TRIGGER, IDENTITY, DAN SEQUENCE

Pendahuluan

A. Deskripsi

Pada sistem basis data modern, penyimpanan data tidak hanya berfungsi sebagai tempat menyimpan informasi, tetapi juga mampu menjalankan logika bisnis secara otomatis. SQL Server menyediakan berbagai objek database yang memungkinkan proses tersebut dilakukan langsung di dalam database, sehingga integritas data tetap terjaga tanpa bergantung sepenuhnya pada aplikasi. Tiga objek penting yang sering digunakan adalah IDENTITY, SEQUENCE, dan TRIGGER.

Ketiga fitur tersebut banyak digunakan dalam sistem informasi seperti rumah sakit, perbankan, pergudangan, sekolah, maupun aplikasi penjualan karena mampu meningkatkan otomatisasi, menjaga konsistensi data, serta mempermudah proses audit.

Pada modul ini, pembahasan dilakukan menggunakan database DB_Petshop sehingga mahasiswa dapat memahami penerapan konsep secara langsung melalui studi kasus nyata.

B. IDENTITY

1. *Pengertian IDENTITY*

Dalam pengelolaan basis data, setiap record harus memiliki identitas yang unik agar dapat dibedakan dari record lainnya. Pada banyak kasus, pemberian nomor identitas secara manual dapat menyebabkan kesalahan seperti nomor ganda (duplicate key), nomor yang terlewat, atau data yang tidak konsisten.

Microsoft SQL Server menyediakan fitur IDENTITY untuk menghasilkan nilai numerik secara otomatis setiap kali sebuah record baru ditambahkan ke dalam tabel. Dengan fitur ini, pengembang tidak perlu lagi menentukan nilai kunci utama (primary key) secara manual karena SQL Server akan mengelolanya secara otomatis.

IDENTITY merupakan property yang diterapkan pada kolom bertipe numerik, seperti INT, BIGINT, SMALLINT, atau TINYINT. Nilai yang dihasilkan bersifat berurutan sesuai dengan nilai awal (seed) dan nilai kenaikan (increment) yang telah ditentukan. Secara umum, sintaks IDENTITY dituliskan sebagai berikut:

IDENTITY(seed, increment)

Keterangan:

1. Seed adalah nilai awal yang akan diberikan pada record pertama.
2. Increment adalah nilai penambahan setiap kali record baru dibuat.

Sebagai contoh:

IDENTITY(1,1)

akan menghasilkan urutan:

1
2
3
4
5
...

Sedangkan:

IDENTITY(100,10)

akan menghasilkan:

100
110
120
130
140
...

2. Tujuan Penggunaan *IDENTITY*

Penggunaan *IDENTITY* bertujuan untuk mengotomatisasi proses pemberian nomor unik pada setiap data yang disimpan.

Beberapa manfaat penggunaan *IDENTITY* antara lain:

1. menghindari duplikasi nomor identitas,
2. mempercepat proses input data,
3. mengurangi kesalahan akibat input manual,
4. menjaga konsistensi data,
5. Mempermudah pengembangan aplikasi.

Sebagian besar aplikasi bisnis modern menggunakan mekanisme penomoran otomatis.

3. *Sintaks IDENTITY*

Sintaks umum

```
CREATE TABLE NamaTabel
(
    NamaKolom INT IDENTITY(seed, increment),
    ...
);
```

4. *Kelebihan dan Kekurangan IDENTITY*

Kelebihan

- Nomor dibuat secara otomatis.
- Mengurangi kesalahan input.
- Mudah digunakan.
- Sangat cepat.
- Cocok sebagai surrogate key.
- Mendukung integritas data.

Kekurangan

- Hanya dapat diterapkan pada satu kolom dalam satu tabel.
- Tidak dapat digunakan bersama oleh beberapa tabel.
- Nomor yang telah digunakan tidak dapat digunakan kembali secara otomatis.
- Jika terjadi penghapusan data, akan muncul celah (gap) pada urutan nomor.

C. SEQUENCE

1. *Pengertian SEQUENCE*

Dalam pengembangan sistem basis data, sering dijumpai kebutuhan untuk menghasilkan nomor urut secara otomatis. Pada materi sebelumnya telah dipelajari bahwa SQL Server menyediakan IDENTITY sebagai mekanisme penomoran otomatis pada suatu kolom. Namun, terdapat kondisi di mana penomoran tidak hanya digunakan oleh satu tabel, tetapi juga harus dapat digunakan bersama oleh beberapa tabel atau sesuai dengan kebutuhan aplikasi.

Untuk memenuhi kebutuhan tersebut, SQL Server menyediakan objek database yang disebut SEQUENCE. SEQUENCE adalah objek database yang menghasilkan deretan nilai numerik secara berurutan dan dapat dipanggil kapan saja menggunakan perintah SQL. Berbeda dengan IDENTITY yang melekat pada sebuah kolom tabel,

SEQUENCE berdiri sebagai objek independen sehingga dapat digunakan oleh satu atau lebih tabel sekaligus.

Karena bersifat independen, SEQUENCE memberikan fleksibilitas yang lebih tinggi dibandingkan dengan IDENTITY, terutama dalam menghasilkan nomor transaksi, nomor reservasi, nomor invoice, atau kode dokumen lainnya.

2. Karakteristik SEQUENCE

Beberapa karakteristik SEQUENCE dalam SQL Server adalah sebagai berikut:

- Merupakan objek database yang berdiri sendiri.
- Tidak terikat pada tabel tertentu.
- Dapat digunakan oleh beberapa tabel secara bersamaan.
- Nilai diperoleh menggunakan perintah NEXT VALUE FOR.
- Nilai dapat diatur mulai dari angka tertentu (START WITH).
- Nilai dapat dinaikkan maupun diturunkan (INCREMENT BY).
- Mendukung batas minimum dan maksimum (MINVALUE dan MAXVALUE).
- Dapat diatur agar kembali ke nilai awal (CYCLE) atau berhenti saat mencapai batas (NO CYCLE).

3. Cara Kerja SEQUENCE

Berbeda dengan IDENTITY yang menghasilkan nomor saat proses INSERT, SEQUENCE menghasilkan nomor saat dipanggil menggunakan fungsi NEXT VALUE FOR. Ilustrasi proses kerja SEQUENCE ditunjukkan pada Gambar 3.1.



Karena tidak bergantung pada tabel tertentu, nilai yang dihasilkan dapat digunakan sesuai kebutuhan aplikasi.

4. Sintaks Dasar SEQUENCE

Sintaks umum pembuatan SEQUENCE adalah sebagai berikut.

```
CREATE SEQUENCE NamaSequence  
AS INT  
START WITH nilai_awal
```

INCREMENT BY kenaikan;

Keterangan:

Bagian	Fungsi
CREATE SEQUENCE	Membuat objek Sequence
AS INT	Menentukan tipe data
START WITH	Menentukan nilai pertama
INCREMENT BY	Menentukan kenaikan setiap nilai

5. Perbedaan *IDENTITY* dan *SEQUENCE*

Meskipun keduanya menghasilkan nomor secara otomatis, terdapat beberapa perbedaan mendasar antara *IDENTITY* dan *SEQUENCE*.

Aspek	IDENTITY	SEQUENCE
Jenis	Properti kolom	Objek database
Terikat tabel	Ya	Tidak
Digunakan banyak tabel	Tidak	Ya
Dipanggil secara manual	Tidak	Ya
Fleksibilitas	Rendah	Tinggi
Cocok untuk	Primary Key	Nomor transaksi, invoice, reservasi

6. Kekurangan dan Kelebihan *SEQUENCE*

Penggunaan *SEQUENCE* memiliki beberapa kelebihan, antara lain:

- Dapat digunakan oleh lebih dari satu tabel.
- Tidak bergantung pada proses INSERT.
- Dapat menghasilkan berbagai format kode bisnis.
- Nilai awal dan kenaikan dapat diatur.
- Dapat digunakan pada Stored Procedure, Function, maupun Trigger.
- Mudah dipadukan dengan fungsi string untuk membentuk kode transaksi.

Selain memiliki kelebihan, *SEQUENCE* juga memiliki beberapa keterbatasan.

- Nilai yang sudah diambil tidak dapat dikembalikan meskipun transaksi dibatalkan.
- Pengembang harus mengatur sendiri format kode yang dihasilkan.
- Penggunaan yang tidak tepat dapat menyebabkan celah nomor (gap).

D. TRIGGER

1. Pengertian Trigger

Dalam sistem basis data modern, sering kali terdapat kebutuhan untuk *menjalankan proses secara otomatis saat terjadi perubahan data*. Sebagai contoh, ketika stok produk berubah, sistem harus mencatat perubahan tersebut ke dalam tabel audit. Ketika data pelanggan dihapus, sistem harus memeriksa apakah pelanggan tersebut masih memiliki transaksi. Proses-proses seperti ini tidak selalu harus dilakukan oleh aplikasi, tetapi dapat dijalankan langsung oleh database.

SQL Server menyediakan objek database yang disebut Trigger untuk memenuhi kebutuhan tersebut. Trigger adalah sekumpulan perintah Transact-SQL (T-SQL) yang akan dijalankan secara otomatis sebagai respons terhadap suatu peristiwa (event) pada tabel atau view. Peristiwa tersebut dapat berupa penambahan data (INSERT), perubahan data (UPDATE), atau penghapusan data (DELETE).

Berbeda dengan Stored Procedure yang dijalankan secara eksplisit oleh pengguna atau aplikasi, Trigger akan dieksekusi secara otomatis oleh SQL Server ketika peristiwa yang ditentukan terjadi.

Dengan demikian, Trigger sangat bermanfaat untuk menjaga konsistensi data, menerapkan aturan bisnis (business rules), melakukan validasi, serta membangun mekanisme audit (audit trail).

2. Karakteristik Trigger

Trigger memiliki beberapa karakteristik penting sebagai berikut.

- Dieksekusi secara otomatis oleh SQL Server.
- Tidak dapat dipanggil secara langsung menggunakan perintah EXEC.
- Selalu dikaitkan dengan tabel atau tampilan.
- Dipicu oleh perintah INSERT, UPDATE, atau DELETE.
- Dapat mengakses tabel virtual yang di-insert dan di-delete.
- Dapat menjalankan satu atau lebih perintah SQL.
- Berjalan dalam transaksi (transaction) yang sama dengan perintah pemicunya.

3. Cara Kerja Trigger

Secara umum, mekanisme kerja Trigger dapat digambarkan sebagai berikut.



Apabila Trigger mengalami kesalahan (error), maka seluruh transaksi dapat dibatalkan (ROLLBACK) sehingga perubahan data tidak disimpan ke dalam database.

4. Sintaks Dasar Trigger

Sintaks dasar Trigger pada SQL Server adalah sebagai berikut.

```
CREATE TRIGGER NamaTrigger
ON NamaTabel
AFTER INSERT
AS
BEGIN
    -- Perintah SQL
END;
```

Keterangan:

Bagian	Fungsi
CREATE TRIGGER	Membuat Trigger baru
ON	Menentukan tabel yang diawasi
AFTER INSERT	Trigger dijalankan setelah proses INSERT
AS	Awal isi Trigger
BEGIN ... END	Blok perintah SQL

5. Jenis Trigger pada SQL Server

Berdasarkan waktu eksekusinya, Trigger dibedakan menjadi dua jenis.

a. AFTER Trigger

AFTER Trigger dijalankan setelah proses INSERT, UPDATE, atau DELETE berhasil dilakukan. Jenis trigger ini paling banyak digunakan karena perubahan data telah berhasil dilakukan sebelum trigger dijalankan.

Contoh penerapan:

- mencatat perubahan stok,
- membuat audit trail,
- mengirim notifikasi,

- memperbarui data lain.

b. **INSTEAD OF Trigger**

INSTEAD OF trigger menggantikan proses INSERT, UPDATE, atau DELETE yang sebenarnya. Dengan kata lain, SQL Server tidak menjalankan perintah asli, tetapi menjalankan isi Trigger terlebih dahulu.

INSTEAD OF Trigger biasanya digunakan pada:

- View yang kompleks,
- validasi data,
- pengendalian hak akses,
- implementasi logika bisnis tertentu.

Pada modul ini, pembahasan difokuskan pada AFTER Trigger karena merupakan jenis Trigger yang paling sering digunakan dalam implementasi sistem informasi.

6. *Event yang Memicu Trigger*

Trigger dapat dipicu oleh 3 jenis operasi DML.

- INSERT Trigger: Dijalankan ketika data baru ditambahkan.
- UPDATE Trigger: Dijalankan ketika data mengalami perubahan.
- DELETE Trigger: Dijalankan ketika data dihapus.

7. *Kekurangan dan Kelebihan Trigger*

Penggunaan trigger memberikan beberapa keuntungan.

- Otomatis: Tidak memerlukan campur tangan pengguna.
- Menjaga Integritas Data: Seluruh pengguna akan mengikuti aturan bisnis yang sama.
- Audit Trail: Seluruh perubahan data dapat direkam secara otomatis.
- Konsisten: Logika bisnis berada di database, sehingga semua aplikasi yang menggunakan database akan menampilkan perilaku yang sama.
- Aman: Trigger dapat mencegah data yang tidak valid masuk ke dalam database.

Kelemahan Trigger

Walaupun sangat bermanfaat, Trigger juga memiliki beberapa kelemahan.

- Sulit Dilacak: Karena berjalan otomatis, pengguna sering tidak menyadari bahwa Trigger sedang berjalan.

- b. Menurunkan performa: Trigger yang terlalu kompleks dapat memperlambat proses INSERT, UPDATE, maupun DELETE.
- c. Sulit Dipelihara: Apabila terdapat banyak trigger dalam satu database, proses pemeliharaan menjadi lebih sulit.
- d. Potensi Trigger Berantai: Satu trigger dapat memicu trigger lain sehingga menghasilkan proses yang panjang dan sulit dianalisis.